Funciones Hash

MATHCON · CURSO BLOCKCHAIN

S1 Qué hace una función hash
S2 Las tres resistencias
S3 Construcción iterada y HMAC
S4 El ataque del cumpleaños
S5 Estándares y roturas
S6 Contraseñas, sal y costo
S7 Árboles de Merkle
S8 Compromisos y blockchain
S9 Matriz de consecuencias
EV Retos y evidencia
AX Anexo · seguridad hoy

Intuición | Formal | Ambos

Modo plano (proyección)

2 / 12 retos resueltos

8 SEMANAS · LICENCIATURA EN SISTEMAS · LAB-IA / MATHCON

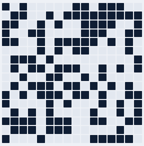
Una huella de *256 bits* para cualquier cosa que exista

Escribe algo abajo. La retícula de la derecha es su hash SHA-256, bit por bit. Cambia una sola letra y observa cuántas celdas se voltean: ése es el punto de partida de todo el curso.

`Satoshi Nakamoto`

`a0dc65ff ca799873 cbea0ac2 74015b95 26505daa aed38515 5425f733 7704883e`

Escribe para comparar contra el digest anterior.



ESPEJO DE BITS · 16 × 16 = 256

SEMANA 1 · MÓDULO 1

Qué hace exactamente una función hash

Objetivo: reconocer las cuatro propiedades observables —determinismo, longitud fija, uniformidad y avalancha— y distinguir un hash criptográfico de uno que no lo es.

Piensa en una licuadora industrial. Metes lo que sea —una palabra, una película, la Biblioteca Nacional— y siempre sale exactamente un litro de puré. Siempre un litro: ni más si metes más, ni menos si metes menos. Y el puré de una manzana nunca se parece al puré de dos manzanas: no hay “casi igual”.

Tres cosas que la licuadora cumple y conviene tener presentes:

- **Es determinista.** La misma manzana, el mismo puré, hoy y en diez años, en tu máquina y en la mía.
- **No se puede des-licuar.** Del puré no recuperas la manzana.
- **Es discontinua a propósito.** Quitarle un pixel a una foto no cambia “un poquito” el resultado: lo cambia por completo. Eso es el *efecto avalancha*, y es la propiedad que hace útil a un hash para detectar alteraciones.

Esa última propiedad es la que estás viendo arriba en la retícula: la mitad de las celdas se voltea aunque sólo toques una tecla.

Laboratorio 1.1 — Medidor de avalancha

BIT-FLIP TEST · AC · SAC · LINEALIDAD

El procedimiento de los apuntes toma k entradas aleatorias, les voltea un bit y mide la distancia de Hamming entre los dos digests; la avalancha completa es $\mathrm{AC} = \frac{\text{bits cambiados}}{\text{bits de salida}}$, y debe rondar el 50 %.

> QUÉ OBSERVAR
>
> Vas a comprobar que **el 50 % no basta**: CRC32 también lo alcanza. Por eso el laboratorio mide tres cosas, y sólo las dos últimas separan de verdad una función criptográfica de una que no lo es.
>
> 1. **AC promedio.** Condición necesaria, no suficiente. Casi cualquier función que mezcle bits la aprueba.
> 2. **Patrones de diferencia.** Se repite *siempre el mismo* volteo de bit sobre mensajes distintos. Si la función es lineal, la diferencia de salida no depende del mensaje y aparece un único patrón, una y otra vez. Un hash criptográfico produce un patrón distinto cada vez.
> 3. **Matriz SAC.** Cada bit de entrada contra cada bit de salida: la celda mide con qué frecuencia voltear el primero voltea el segundo. Todo debe quedarse en 0.5. Las celdas en 0 o en 1 son relaciones deterministas, y cada una es una ecuación que el atacante puede resolver.

Función: SHA-256 | Pruebas: 600 | Medir avalancha

AC PROMEDIO	DESVIACIÓN Σ / ESPERADA	PATRONES DISTINTOS	MÁX. DESVÍO SAC
—	—	—	—

DISTRIBUCIÓN DE BITS CAMBIADOS

Cada barra cuenta cuántas de las pruebas cambiaron esa cantidad exacta de bits. Lo ideal es una campana centrada en la línea punteada, $n/2$, con desviación $\sqrt{n}/2$: eso es lo que produce una binomial $\mathrm{Bin}(n, 1/2)$, es decir, lanzar una moneda por cada bit de salida. Una barra única —toda la masa en un solo valor— significa varianza cero: el número de bits que cambian no depende del mensaje, y eso es el sello de una función lineal.

Cómo leer la matriz de abajo

Imagina **32 interruptores** —los primeros 32 bits del mensaje, una fila cada uno— y **n focos**, los bits del digest, una columna cada uno. La celda de la fila i y la columna j responde a una sola pregunta: *si acciono el interruptor i, ¿con qué frecuencia reacciona el foco j?* Se mide accionándolo sobre mensajes aleatorios distintos y contando.

Una función ideal responde **lanzando una moneda por cada foco**: cada uno cambia la mitad de las veces, sin importar qué interruptor toques. Eso es toda la matriz en verde. Los dos extremos son los que delatan estructura, y significan cosas opuestas:

- **Rojo — el foco nunca reacciona (frecuencia 0).** Ese bit de salida es sordo a ese bit de entrada: el atacante puede modificar el mensaje ahí sabiendo de antemano que esa parte del digest no se moverá.
- **Azul — el foco siempre reacciona (frecuencia 1).** Está cableado directamente al interruptor: lo copia. El atacante sabe exactamente qué bit del digest voltear y cómo compensarlo.
- **Verde — frecuencia ≈ 0.5.** No hay relación que explotar; la única forma de saber

qué pasa con ese foco es calcular el hash.

Rojo y azul son igual de graves: ambos son ecuaciones deterministas entre entrada y salida. El verde es, literalmente, *la ausencia de ecuaciones*. Y aquí está el detalle que el promedio esconde: **CRC32 llena la matriz de rojo y azul casi a partes iguales** —por eso su AC promedia un impecable 50 %— y no tiene una sola celda verde. El promedio miente; la matriz no.

MATRIZ SAC — FILAS: BITS DE ENTRADA 0…31 · COLUMNAS: BITS DE SALIDA 0…N-1

🟥 0 · nunca cambia 🟩 ≈ 0.5 · como debe ser 🟦 1 · siempre cambia

Pasa el puntero sobre la matriz para leer una celda concreta.

Ejecuta la medición con SHA-256 y luego con CRC32. El primer indicador dirá casi lo mismo; los otros dos, todo lo contrario.

Laboratorio 1.2 — Uniformidad: dónde caen los mensajes

DISTRIBUCIÓN EN CUBETAS

Se hashean miles de cadenas *muy parecidas entre sí* (`usuario0`, `usuario1`, …) y se reparten en 64 cubetas según los últimos bits del digest. Cada barra es una cubeta y su altura, cuántos mensajes cayeron ahí. Un hash criptográfico las esparce; CRC32 y las funciones de suma revelan estructura.

Las barras no deben salir iguales. Con N muestras en 64 cubetas se esperan $N/64$ por cubeta, con una fluctuación natural de $\pm\sqrt{N/64}$ —para 4000 muestras, 62.5 ± 7.9, o sea un dentado de ±13 %—. El χ^2 lo cuantifica: bajo uniformidad su valor esperado es igual a los grados de libertad, 63, con desviación $\sqrt{126} \approx 11$, y se rechaza por encima de 82.5. Un χ^2 cercano a cero sería tan sospechoso como uno de 300: significaría un repartidor demasiado regular, un contador disfrazado de hash.

Ojo con lo que *no* prueba esta gráfica: mira sólo 6 bits del digest y la aprueba cualquier mezclador decente, incluido el toy-hash de una sola ronda. La uniformidad es condición necesaria y barata. La que separa de verdad es la matriz del laboratorio anterior.

Función SHA-256 ▾ Muestras 4000 Repartir

SEMANA 2 · MÓDULO 1

Las tres resistencias, y por qué no son la misma

Objetivo: formular con precisión resistencia a preimagen, a segunda preimagen y a colisión; demostrar la implicación que sí vale y construir el contraejemplo de la que no.

Tres preguntas distintas, tres dificultades distintas:

- **Preimagen.** Te doy una huella dactilar y te pido encontrar al dueño. *"Aquí está el hash: dime qué lo produjo."*
- **Segunda preimagen.** Te doy a una persona concreta y te pido encontrar a otra con su misma huella. *"Este contrato ya está firmado: fabrica otro contrato con el mismo hash."*
- **Colisión.** Te pido encontrar *cualesquiera* dos personas que compartan huella. *"Fabrica dos documentos, los que tú quieras, con el mismo hash."*

La tercera es la más fácil para el atacante, porque puede escoger ambos lados. Y por eso es la primera que cae: MD5 y SHA-1 están rotos *en colisión*, no en preimagen.

> **PROPOSICIÓN 1**
>
> Si H es resistente a colisiones, entonces es resistente a segunda preimagen.

Demostración por contrapuesta VER DEMOSTRACIÓN

> **PROPOSICIÓN 2**
>
> El recíproco falla en el sentido fuerte: existe h resistente a colisiones que *no* es resistente a preimagen.

Construcción del contraejemplo VER DEMOSTRACIÓN

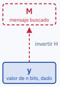
Los tres problemas, uno junto a otro

Lo único que distingue a las tres definiciones es *qué recibe el adversario* y *qué queda libre para que él lo elija*. Ese reparto es todo el contenido matemático, y es lo que codifica el color:

☐ dato que recibe el adversario ⬚ lo que debe encontrar ▢ valor libre, sin restricción

Resistencia a preimagen

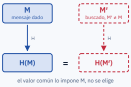
ONE-WAY · COSTO IDEAL 2^n



Dado $y \in \{0,1\}^n$, hallar cualquier M con $H(M) = y$. El adversario no elige nada del enunciado: sólo el mensaje.

Resistencia a 2ª preimagen

COSTO IDEAL 2^n

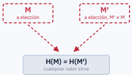


el valor común lo impone M, no se elige

Dado M, hallar $M' \neq M$ con $H(M') = H(M)$. El objetivo está fijado de antemano: por eso es más difícil que una colisión.

Resistencia a colisiones

COSTO IDEAL $2^{n/2}$ · CUMPLEAÑOS



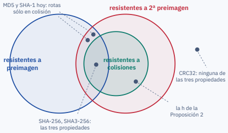
nada está fijado: el adversario elige los dos lados

Hallar cualquier par $M \neq M'$ con $H(M) = H(M')$. Al no fijarse ni el par ni el valor común, la raíz cuadrada del espacio basta.

Qué se corrigió respecto de las figuras de los apuntes VER DEMOSTRACIÓN

El diagrama de relaciones

Cada región es un conjunto de funciones hash: las que cumplen esa propiedad. Del par de proposiciones anteriores se sigue exactamente esta figura — colisión ⊂ segunda preimagen por la Proposición 1, mientras que preimagen se solapa sin contener por la Proposición 2.


MD5 y SHA-1 hoy: rotas sólo en colisión
resistentes a 2ª preimagen
resistentes a preimagen
resistentes a colisiones
CRC32: ninguna de las tres propiedades
la h de la Proposición 2
SHA-256, SHA3-256: las tres propiedades

La h construida en la Proposición 2 vive fuera del círculo azul y dentro del verde: eso es justo lo que prueba que la contención inversa falla. Ninguna región del diagrama está vacía. Conviene detenerse en que **MD5 y SHA-1 comparten región** pese a la enorme distancia entre ellas: MD5 colisiona en segundos y SHA-1 costó $2^{63.1}$, pero en ninguna de las dos se conoce un ataque práctico de preimagen —el mejor contra MD5 cuesta $2^{123.4}$—, y por eso ambas siguen dentro del azul. Quien de verdad queda fuera de los tres círculos es CRC32: invertible en tiempo lineal, con segunda preimagen y colisiones triviales. Es una suma de comprobación, no una función hash criptográfica.

Laboratorio 2.1 — Caza de colisiones sobre una función de juguete

N AJUSTABLE · 8–28 BITS

Con SHA-256 nadie de esta clase encontrará una colisión, hoy ni en un millón de años de cómputo. La seguridad no es magia: es tamaño. Reduce n y compruébalo tú mismo.

Bits de salida n = 16

Buscar: una colisión (cualquier par)

Atacar | Detener

INTENTOS	PREDICCIÓN TEÓRICA	TIEMPO
0	≈ 301 (2^8.0)	—

```
Sin resultados todavía.
```

Observa el contraste al cambiar de modo: la colisión aparece cerca de $2^{n/2}$ intentos; la preimagen exige 2^n. Con $n = 24$, son 4 mil frente a 16 millones. Ésa es toda la diferencia entre las semanas 2 y 4.

SEMANA 3 · MÓDULO 2

Cómo está construida por dentro, y qué se rompe por eso

Objetivo: entender la construcción iterada de Damgård–Merkle, ver SHA-256 ronda por ronda, y descubrir el ataque de extensión de longitud —la razón de existir de HMAC.

Un hash no procesa el mensaje completo de golpe: lo corta en bloques de 64 bytes y los va digiriendo uno tras otro. Cada bloque se mezcla con el resultado del anterior, como una masa que se amasa por tandas. El estado final, después del último bloque, es el digest.

Esa arquitectura tiene una consecuencia incómoda que casi nadie ve venir: **si conoces el digest, conoces el estado interno con el que quedó la máquina**. Y con el estado en la mano puedes seguir amasando —añadir texto al final del mensaje— sin conocer el principio. Lo verás funcionar más abajo.

Laboratorio 3.1 — Microscopio de SHA-256

64 RONDAS · ESTADO A…H

Aquí no hay caja negra. Se muestra el relleno real del mensaje, la expansión del bloque a 64 palabras W_i, y la evolución del estado interno ronda por ronda. Arrastra la ronda y observa cómo la difusión se completa mucho antes de terminar.

```
abc
```

Ronda 64 de 64

Bloque: 1 de 1

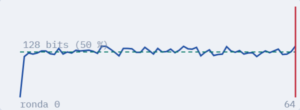
ESTADO INTERNO (8 PALABRAS DE 32 BITS)

```
a = 506e3058 b = d39a2165 c = 04d24d6c d = b85e2ce9
e = 5ef50f24 f = fb121210 g = 948d25b6 h = 961f4894
```

BLOQUE RELLENADO (512 BITS) Y PALABRA W_i

```
W63 = 12b1edeb
bloque (16 palabras iniciales):
61626380 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000018
```

DIFUSIÓN ACUMULADA: BITS DEL ESTADO QUE YA CAMBIARON RESPECTO A LA RONDA 0



En la ronda 64, 145 de los 256 bits del estado difieren del valor inicial (56.6 %). La difusión completa se alcanza alrededor de la ronda 2: las 64 rondas son un margen de seguridad deliberado.

Descripción de las funciones de SHA256:

Laboratorio 3.2 — Falsificar un MAC sin conocer la clave

LENGTH EXTENSION ATTACK · PASO A PASO

Un servidor autentica peticiones con $t = H(k \,\|\, m)$, creyendo que sin k nadie puede producir un t válido. Tú eres el atacante: ves m y t, nunca la clave. El recorrido muestra los bytes exactos en cada etapa — los que el servidor tiene, los que tú puedes calcular y los que nunca necesitas.

Clave del servidor `clave-secreta`
Mensaje m `usuario=ana&admin=no`
Texto añadido `&admin=si`

Ejecutar el ataque paso a paso · Cargar ejemplo de API · ☐ el atacante no conoce la longitud de la clave

☐ clave: el atacante nunca la ve · ☐ mensaje original · ☐ relleno · ☐ longitud en bits · ☐ texto añadido por el atacante

Laboratorio 3.3 — HMAC: la construcción correcta

RFC 2104

$\mathrm{HMAC}_k(m) = H\big((k \oplus \mathrm{opad}) \,\|\, H((k \oplus \mathrm{ipad}) \,\|\, m)\big)$. El doble anidamiento hace que el estado interno final no sea el de un prefijo del mensaje, y la extensión de longitud muere ahí.

Clave `clave`
Mensaje `transferir 100 a Bob`

H(K ‖ M) — VULNERABLE

```
83cc0ca05435a731822e8f027b7788badb99d2f3ce9d84ac4d7fa3ff
a63e50a7
```

HMAC-SHA256(K, M) — CORRECTO

```
2cc578bb28d02662c2e203031e26ee7121b54187733d22680ecaa077
482d3b29
```

Cambia un carácter de la clave y observa que ambas salidas cambian por completo. La diferencia no está en la apariencia del digest, sino en que el de la izquierda permite extender el mensaje sin conocer la clave y el de la derecha no.

SEMANA 4 · MÓDULO 3

El ataque del cumpleaños: por qué 256 bits valen 128

Objetivo: derivar la cota $\approx 1.1774\sqrt{N}$, verificarla experimentalmente y usarla para dimensionar la seguridad real de cualquier función hash.

En un salón con 23 personas, la probabilidad de que dos compartan cumpleaños supera el 50 %. La intuición falla porque uno se pregunta por sí mismo —“¿alguien cumple el mismo día que yo?”— cuando la pregunta real es por *todos los pares posibles*, y con 23 personas hay 253 pares.

Traducido a hashes: no cuesta 2^{256} romper SHA-256 por colisión, cuesta 2^{128}. La raíz cuadrada es el precio de que el atacante pueda elegir ambos mensajes.

Laboratorio 4.1 — La paradoja, en personas y en bits

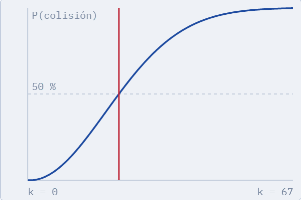
EXACTO VS. APROXIMACIÓN

Espacio: `365 días del año`
Elementos k = 23

PROBABILIDAD EXACTA
50.730 %

APROXIMACIÓN $1 - E^{-K(K-1)/2N}$
50.000 %

K PARA 50 %
22



PERSONAS	PROBABILIDAD DE COLISIÓN
10	11.69 %
23	50.73 %
30	70.63 %
50	97.04 %
70	99.92 %
100	100.00 %

Laboratorio 4.2 — Verificación experimental de $1.1774\sqrt{N}$

MONTE CARLO SOBRE TOY-HASH

Se ejecuta el ataque del cumpleaños real —hasheando cadenas aleatorias y guardándolas en una tabla hasta que dos coincidan— y se repite muchas veces. Compara la media empírica de intentos con la predicción teórica.

n = 20 bits
Repeticiones `200`
Ejecutar experimento

MEDIA EMPÍRICA
—

TEORÍA 1.1774·2^{N/2}
—

ERROR RELATIVO
—

Histograma de intentos hasta la primera colisión, con la media empírica (azul) y la predicción teórica (verde) superpuestas. La distribución es asimétrica: la mediana cae por debajo de la media, otro detalle que la fórmula del 50 % explica.

SEMANA 5 · MÓDULO 4

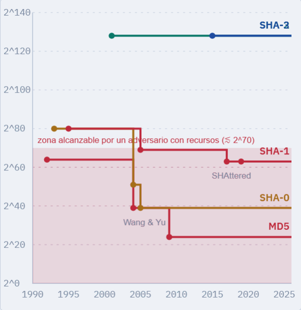
Cuáles usar hoy, cuáles ya cayeron, y cuándo

Objetivo: leer la historia de las roturas como una carrera entre la cota del cumpleaños y los ataques diferenciales; saber qué estándar elegir y justificarlo.

Ninguna función hash muere de golpe. Primero alguien publica un ataque que cuesta un poco menos que la fuerza bruta —nadie se alarma—, luego el costo baja lo suficiente para un laboratorio, y un día aparecen dos archivos distintos con el mismo hash, descargables por cualquiera. En ese momento la función ya llevaba años muerta en los papers.

■ Línea de tiempo del deterioro

Cada barra es el $\log_2$ del costo de la mejor colisión conocida. La franja sombreada marca lo que un adversario bien financiado puede pagar hoy.



Laboratorio 5.1 — Comparador de algoritmos LONGITUD · VELOCIDAD · ESTATUS

MathCon

Medir velocidad

FUNCIÓN	BITS	DIGEST	ESTATUS	MB/S
MD5 colisiones en segundos	128	`8b346d409683c5c206a3dac4cd1466f3`	ROTO	–
SHA-1 colisión pública desde 2017	160	`707ad6211d44184e9cbeff26c97ee1e0…`	ROTO	–
SHA-256 estándar de Bitcoin y TLS	256	`119ce07e092625e03855d8b404e10a49…`	VIGENTE	–
SHA3-256 esponja Keccak, FIPS 202	256	`37cf293428b689bd8cf3b1db9f4edece…`	VIGENTE	–
CRC32 detección de errores, nunca seguridad	32	`b36b8675`	NO CRIPTOGRÁFICA	–

La velocidad se mide con un bloque de datos aleatorios y tamaño adaptativo, usando las implementaciones didácticas en JavaScript de esta página: sirve para comparar entre sí, no como cifra absoluta. SHA3-256 sale especialmente penalizado porque su permutación Keccak se implementó aquí con enteros de 64 bits emulados.

Laboratorio 5.2 — Dos archivos distintos, el mismo hash VERIFICACIÓN LOCAL · NADA SE SUBE

Arrastra dos archivos y compáralos con MD5, SHA-1 y SHA-256 a la vez. Para ver una colisión real, descarga el par de PDF de shattered.io: son visualmente distintos y comparten SHA-1. Todo el cálculo ocurre en tu equipo; ningún byte sale del navegador.

Archivo A Seleccionar archivo Sin archivos seleccionados

–

Archivo B Seleccionar archivo Sin archivos seleccionados

–

SEMANA 6 · MÓDULO 3

Contraseñas: por qué un hash rápido es un defecto

Objetivo: cuantificar el costo de un ataque por diccionario, entender qué protege la sal y qué protege el factor de trabajo, y por qué SHA-256 es la elección equivocada para almacenar contraseñas.

Guardar `SHA-256(contraseña)` parece prudente: si roban la base, no hay contraseñas en claro. Pero el atacante no necesita invertir el hash. Le basta con hashear las diez mil contraseñas más comunes y buscar coincidencias — y como el hash es determinista, **una sola tabla sirve para todos los usuarios de todas las bases de datos del mundo**. Eso es una tabla arcoíris.

La sal rompe la reutilización: un valor aleatorio distinto por usuario obliga a rehacer la tabla para cada uno. El factor de trabajo ataca lo otro: hace que cada intento cueste, digamos, 100 ms en lugar de 0.000002 s.

Laboratorio 6.1 — Tabla arcoíris contra sal ATAQUE POR DICCIONARIO

Ocho usuarios ficticios eligieron contraseñas de un diccionario de uso común. Precomputa la tabla y observa cuántas cuentas caen; luego activa la sal y repite exactamente el mismo ataque.

☐ usar sal aleatoria por usuario Diccionario: 91 candidatos **Ejecutar ataque**

CUENTAS COMPROMETIDAS	HASHES CALCULADOS	TIEMPO
—	—	—

La sal no impide adivinar una contraseña débil concreta: impide atacar a todos a la vez con el mismo trabajo. Fíjate en la columna de hashes calculados, no en la de cuentas rotas.

Laboratorio 6.2 — El precio de una verificación PBKDF2 REAL · WEBCRYPTO

Mide el tiempo real de derivar una clave con distinto número de iteraciones y extrapola cuánto tardaría un atacante en probar un diccionario completo.

Iteraciones `100000` Tamaño del diccionario `1 millón` **Medir**

POR VERIFICACIÓN	DICCIONARIO COMPLETO (1 NÚCLEO)	CON 10 000 GPU ($\times 10^5$)
—	—	—

PBKDF2 es el único KDF disponible de forma nativa en el navegador; bcrypt, scrypt y Argon2 son preferibles en un servidor real porque también imponen costo en memoria. La comparación de órdenes de magnitud, sin embargo, es la misma. Ese costo en memoria es exactamente el mismo mecanismo que estudiaremos en la semana 8 para frenar a los ASIC de minado: por qué unas funciones se ejecutan en masa y otras no.

SEMANA 7 · MÓDULO 5

Árboles de Merkle: probar pertenencia sin descargar todo

Objetivo: construir un árbol de Merkle con doble SHA-256 como Bitcoin, generar y verificar una prueba de inclusión, y calcular el ahorro logarítmico que hace posible la verificación simplificada de pagos.

Un bloque de Bitcoin puede contener miles de transacciones. Tu teléfono no va a descargar 600 GB para comprobar que te pagaron. El árbol de Merkle resuelve exactamente eso: hashea las transacciones por parejas, luego los resultados por parejas, y así hasta quedarse con un único valor —la *raíz*— que resume todo el conjunto.

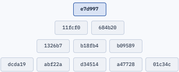
Lo elegante es la prueba: para convencerte de que tu transacción está adentro, sólo necesito darte los hashes hermanos del camino hasta la raíz. Con 1 048 576 transacciones son 20 hashes: 640 bytes en lugar de gigabytes.

Laboratorio 7.1 — Constructor de árbol y prueba de inclusión DOBLE SHA-256 · SPV

Transacciones, una por línea

```
Tra1: Alice paga 2 BTC a Bob
Tra2: Bob paga 0.5 BTC a Carol
Tra3: Carol paga 1 BTC a Dave
Tra4: Dave paga 0.1 BTC a Erin
Tra5: Erin paga 3 BTC a Alice
```

Construir árbol Probar inclusión de `Tra1: Alice paga 2 BTC` **Generar prueba** **Alterar esa transacción**



RAÍZ DE MERKLE

`e7d9978a2bc34f5b84cd8a1de4e4284610d945d948efc90a95e638ce054455eb`

HOJAS	ALTURA DEL ÁRBOL	TAMAÑO DE LA PRUEBA
5	3	96 bytes

En verde, el camino que recomputa el verificador; en ámbar, los hermanos que forman la prueba. Al alterar una hoja, en rojo, se propaga la invalidación hasta la raíz: exactamente lo que detecta un nodo completo.

Laboratorio 7.2 — El ahorro logarítmico POR QUÉ ESTO ESCALA

Transacciones en el bloque: 4,096

DESCARGAR TODO	PRUEBA DE MERKLE	FACTOR DE AHORRO
0.98 MB	634 bytes	×1,615

SEMANA 8 · MÓDULO 5

Compromisos, prueba de trabajo y la cadena

Objetivo: usar un hash como esquema de compromiso con las propiedades de ocultamiento y vinculación; entender la prueba de trabajo como un problema de preimagen parcial; y ver por qué encadenar hashes vuelve inmutable un registro.

Un compromiso es un sobre cerrado: pones dentro tu decisión, entregas el sobre, y más tarde lo abres. Quien lo tiene no puede leerlo (*ocultamiento*) y tú ya no puedes cambiar lo que hay dentro (*vinculación*). Un hash hace las dos cosas a la vez… siempre que recuerdes meter un valor aleatorio junto con tu decisión.

Laboratorio 8.1 — El volado por teléfono
OCULTAMIENTO Y VINCULACIÓN

Alice y Bob deciden algo a distancia sin poder confiar el uno en el otro. Ejecuta el protocolo paso a paso; luego quita el valor aleatorio y observa cómo Bob rompe el compromiso al instante.

Alice elige `águila` ☑ incluir valor aleatorio r **Fase 1: comprometer** Fase 2: revelar Bob intenta adivinar Detener

Laboratorio 8.2 — Prueba de trabajo y cadena de bloques
PREIMAGEN PARCIAL

Minar es resolver $H(\text{bloque} \,\|\, \text{nonce}) < \text{objetivo}$: una preimagen parcial, el único problema difícil de esta lista que se resuelve por fuerza bruta a propósito. Cada bloque incluye el hash del anterior, así que alterar uno invalida todos los siguientes.

Dificultad: 4 ceros hexadecimales **Minar la cadena** Detener

BLOQUE 0 · SIN MINAR	BLOQUE 1 · SIN MINAR	BLOQUE 2 · SIN MINAR	BLOQUE 3 · SIN MINAR
DATOS `Bloque génesis`	DATOS `Alice → Bob: 5 BTC`	DATOS `Bob → Carol: 2 BTC`	DATOS `Carol → Dave: 1 BTC`
NONCE 0	NONCE 0	NONCE 0	NONCE 0
HASH PREVIO 000000000000000000…	HASH PREVIO –	HASH PREVIO –	HASH PREVIO –
HASH –	HASH –	HASH –	HASH –

Edita el contenido de cualquier bloque y observa la cascada de invalidación. Para volver a hacer válida la cadena tienes que reminar ese bloque *y todos los posteriores*: eso es el costo de reescribir la historia.

Laboratorio 8.3 — Compromiso de Pedersen: sumar sin revelar
HOMOMORFÍA ADITIVA · CURVA PEQUEÑA

Un compromiso con hash oculta, pero no permite operar. El compromiso de Pedersen $C(v, r) = vG + rH$ sobre una curva elíptica sí: la suma de compromisos es el compromiso de la suma, y eso permite verificar que una transacción no crea dinero de la nada sin conocer los montos.

Entradas `7, 5` Salidas `10, 2` **Verificar sin revelar**

Se usa una curva didáctica sobre $\mathbb{F}_p$ con $p = 1009$ para que los puntos sean legibles; Bitcoin y Monero usan secp256k1 y Ed25519. El emisor elige los factores de cegado de modo que $\sum r_{\text{in}} = \sum r_{\text{out}}$; entonces $\sum C_{\text{in}} - \sum C_{\text{out}} = \left(\sum v_{\text{in}} - \sum v_{\text{out}}\right)G$, que es el punto al infinito *si y sólo si* los montos cuadran. Ningún monto individual se revela en ningún momento.

Por qué SHA-256 se ejecuta en masa y otras funciones no

El laboratorio anterior mina en tu navegador a unos pocos miles de hashes por segundo. Un equipo comercial de minado de 2026 ronda los 580 TH/s con una eficiencia cercana a 9.5 joules por terahash: **en un segundo calcula lo que un núcleo de tu procesador tardaría más de medio año en calcular**. Esa asimetría no es un accidente de fabricación; es una propiedad de SHA-256, y hay funciones hash diseñadas específicamente para no tenerla.

La razón es que SHA-256 es el sueño de un diseñador de circuitos: una secuencia fija de 64 rondas de sumas, rotaciones y XOR sobre 256 bits de estado. Sin memoria, sin bifurcaciones, sin datos impredecibles. Un ASIC despliega las 64 rondas en una cadena de segmentación y entrega un hash por ciclo de reloj, sin leer instrucciones ni decodificarlas. Una CPU dedica la mayor parte de sus transistores a caché, predicción de saltos y ejecución fuera de orden — todo eso, aquí, es silicio desperdiciado.

De ahí sale la idea que define a las funciones resistentes a ASIC: *si el cuello de botella no es la aritmética sino la memoria, el fabricante de circuitos pierde su ventaja*. Nadie puede fabricar celdas de DRAM sustancialmente más baratas o más rápidas que las que Samsung o Micron ya le venden a cualquiera. La memoria es una mercancía; el silicio lógico a medida, no.

Qué defiende a cada función del hardware dedicado

FUNCIÓN	RECURSO QUE IMPONE	MEMORIA POR EVALUACIÓN	VENTAJA DEL HARDWARE DEDICADO	USO
SHA-256, SHA-3, BLAKE2 `ninguna defensa`	Sólo aritmética de registros; ninguna dependencia de datos.	`≈ 0 · unos cientos de bytes de estado`	10^4–10^5× en energía por hash frente a una CPU	Bitcoin, firmas, integridad, Merkle. Aquí la velocidad es virtud.
bcrypt `Provos–Mazières, 1999`	Cajas S de 4 KiB modificadas durante el cálculo, con accesos pequeños y aleatorios.	`4 KiB`	GPU muy penalizada; FPGA y ASIC viables	Contraseñas. No es *memory-hard* en sentido moderno, pero fue el primer diseño hostil al paralelismo masivo.
scrypt `Percival, 2009`	Memoria parametrizable con accesos dependientes de los datos; introduce la métrica AT.	`128 KiB en Litecoin · ≥16 MiB recomendado`	ASIC comercial desde 2014 *con el parámetro pequeño*	Contraseñas, Litecoin. La lección es que el parámetro pesa más que el algoritmo: 128 KiB caben de sobra en un chip.
Argon2id `PHC 2015 · RFC 9106`	Memoria + multiplicaciones de 64 bits que alargan el circuito + resistencia a compromisos.	`64 MiB o 2 GiB según la configuración`	REDUCIDA al controlador de memoria	Contraseñas y derivación de claves. Recomendación vigente: Argon2id con t=1, p=4, m=2 GiB; o t=3, p=4, m=64 MiB si hay poca memoria.
Ethash `Ethereum hasta 2022`	Lecturas aleatorias sobre un DAG de más de 1 GiB: el límite es el ancho de banda de memoria.	`> 1 GiB, creciente con el tiempo`	ASIC desde 2018, pero con ventaja modesta sobre una GPU	Ethereum antes de la transición a prueba de participación. Un ASIC de Ethash es, esencialmente, una GPU sin pantalla.
Equihash `Biryukov–Khovratovich, 2016`	Problema del cumpleaños generalizado (algoritmo de Wagner): el costo está en la tabla, no en el hash.	`≈ 150 MiB en el parámetro (200,9)`	ASIC desde 2018	Zcash. Verificación muy barata: la solución se comprueba en microsegundos.
CryptoNight `Monero hasta 2019`	Scratchpad de 2 MiB dimensionado para caber en la caché L3 de una CPU.	`2 MiB`	ASIC en 2018 → bifurcaciones repetidas de la red	Monero. Caso de estudio del fracaso: 2 MiB también caben en un chip dedicado.
RandomX `Monero desde nov. 2019`	Ejecuta **programas aleatorios** en una máquina virtual — enteros, punto flotante, saltos— encadenando 8 de ellos, sobre un dataset de ~2 GiB.	`2080 MiB · 256 MiB en modo ligero`	MÍNIMA: el ASIC tendría que ser una CPU	Monero. El dataset se deriva con Argon2d de datos de la cadena, de modo que el minero no puede elegir la semilla.

LA IDEA EN UNA FRASE

Las funciones inmunes al hardware dedicado no lo son por ser *más* criptográficas, sino por exigir un recurso que el atacante compra al mismo precio que todos: memoria, latencia y flexibilidad. RandomX lleva el argumento a su conclusión lógica — para construir el circuito óptimo habría que construir un procesador de propósito general, que es precisamente lo que ya está en la computadora de cualquiera.

Laboratorio 8.4 — La memoria como igualador

PRODUCTO ÁREA × TIEMPO

El atacante no está limitado por lo listo que sea, sino por lo que quepa en su presupuesto. Fija cuánta DRAM puede pagar y compara cuántas evaluaciones simultáneas consigue con cada función.

Función `Argon2id RFC, opción baja — 64 MiB` Presupuesto de memoria: 64 GiB

EVALUACIONES EN PARALELO	DRAM PARA UN MILLÓN EN PARALELO	MEMORIA POR EVALUACIÓN
1,024	61 TiB	64 MiB

Con 64 GiB de memoria, un atacante ejecuta **1,024** evaluaciones de **Argon2id RFC, opción baja** a la vez. Añadir silicio lógico no le sirve de nada: para duplicar el paralelismo tiene que duplicar la DRAM, que compra al mismo precio de mercado que el defensor. Compara con la primera opción de la lista, donde la cifra no tiene tope. El modelo es de orden de magnitud: ignora el ancho de banda, la latencia y el coste de los controladores, que sólo empeoran la posición del atacante.

La objeción que conviene discutir en clase

La resistencia a ASIC no es una victoria permanente ni un consenso técnico. Tres reservas honestas:

- **Ninguna función es inmune, sólo cara de atacar.** Toda la lista anterior salvo RandomX terminó con hardware dedicado en el mercado. Lo que cambia no es la existencia del ASIC, sino la magnitud de su ventaja: de cinco órdenes de magnitud a un factor de dos o tres.
- **Puede ser contraproducente.** Si la red se mina con hardware de consumo, ese hardware se *renta*. Varias cadenas pequeñas minables con GPU sufrieron ataques del 51 % contratando capacidad por horas; un parque de ASIC, en cambio, es un costo hundido que sólo vale algo mientras esa cadena concreta siga siendo valiosa. Hay un argumento serio de alineación de incentivos a favor de los ASIC.
- **Se paga en complejidad.** Una prueba de trabajo que ejecuta programas aleatorios en una máquina virtual tiene una superficie de ataque incomparablemente mayor que sesenta y cuatro rondas de sumas y rotaciones. Simplicidad y auditabilidad también son propiedades de seguridad.

Para el caso de las contraseñas de la semana 6, en cambio, el debate no existe: ahí el defensor fija los parámetros, nadie necesita verificar barato desde fuera, y la memoria es una defensa sin contrapartida. Argon2id es la recomendación actual sin matices.

SÍNTESIS · TRANSVERSAL A LAS OCHO SEMANAS

Qué se rompe exactamente cuando falla cada propiedad

Objetivo: leer cada propiedad de seguridad como el soporte de una pieza concreta del sistema —la firma, la raíz de Merkle, el compromiso, el minado— y poder nombrar el ataque que aparece en el momento en que esa propiedad deja de valer.

Hasta aquí las propiedades se enunciaron por separado. Conviene ahora invertir la pregunta: ***si mañana alguien publica un algoritmo que rompe la resistencia a colisión de la función que estás usando, ¿qué deja de funcionar y qué sigue en pie?*** La respuesta no es «todo»: cada propiedad sostiene piezas distintas, y esa asignación es lo que decide qué hay que migrar con urgencia y qué puede esperar.

El caso canónico es la firma digital. Nadie firma un documento: se firma su hash. Si el atacante consigue dos textos con la misma huella —uno inocuo y otro fraudulento— basta con hacer firmar el primero para quedarse con una firma válida sobre el segundo. **La clave privada nunca se compromete y, aun así, la firma es falsificable.** Ésa es la consecuencia exacta de perder la resistencia a colisión.

LA REGLA QUE ORDENA LA TABLA

Una propiedad rota no derriba el sistema entero: derriba *los elementos cuya seguridad se reduce a ella*. Preimagen protege lo que debe permanecer oculto; segunda preimagen protege lo que ya fue publicado y aceptado; colisión protege lo que será elegido por el propio adversario.

Tabla 1 · Cada propiedad y lo que sostiene

La columna central es la operativa: describe lo que un adversario *gana*, no lo que la definición prohíbe.

PROPIEDAD DE SEGURIDAD	ENUNCIADO	SI FALLA, EL ADVERSARIO PUEDE…	DÓNDE DUELE PRIMERO	COSTO IDEAL
Resistencia a preimagen `one-way`	Dado y, es inviable hallar x con $H(x) = y$.	Invertir huellas publicadas: recuperar contraseñas de un volcado, reconstruir el guion de una dirección, **leer el contenido de un compromiso antes de que lo abran**.	Ocultamiento del compromiso · contraseñas · direcciones P2SH	2^n
Resistencia a segunda preimagen `target-collision`	Dado x, es inviable hallar $x' \neq x$ con $H(x') = H(x)$.	Sustituir un objeto *ya publicado y aceptado por otros* sin alterar ningún hash previamente difundido.	Encadenamiento de bloques · prueba de Merkle · documento ya firmado	2^n
Resistencia a colisión `collision-free`	Es inviable hallar cualquier par $x \neq x'$ con $H(x) = H(x')$.	Fabricar de antemano dos objetos intercambiables: **hacer firmar uno y exhibir el otro**, o abrir un mismo compromiso de dos maneras.	Firma digital · vinculación del compromiso · raíz de Merkle	$2^{n/2}$
Resistencia a colisión de prefijo elegido `chosen-prefix`	Dados $P_1 \neq P_2$ fijados por terceros, es inviable hallar S_1, S_2 con $H(P_1 \| S_1) = H(P_2 \| S_2)$.	Dar a las dos versiones un encabezado realista y elegido de antemano —nombres, montos, campos de un certificado X.509—, no dos archivos basura.	Firmas sobre documentos con formato · certificados · PGP	$\geq 2^{n/2}$
Inmunidad a extensión de longitud `propiedad de la construcción`	De $H(m)$ y $\lvert m \rvert$ no debe poder computarse $H(m \| \mathrm{pad} \| z)$ sin conocer m.	Forjar etiquetas de autenticación para mensajes que nunca vio, **sin conocer la clave**, si el MAC es el ingenuo $H(k \| m)$.	MAC artesanal · APIs firmadas · derivación de claves	–
Salida indistinguible de aleatoria `avalancha · uniformidad · PRF`	Ningún bit de salida debe ser predecible ni estar sesgado respecto de la entrada.	Sesgar o anticipar valores derivados del hash: el nonce ganador, el resultado de un sorteo, la clave hija de un monedero jerárquico.	Prueba de trabajo · aleatoriedad pública · HMAC y BIP32	–

Costo de evaluación controlable `sólo para KDF`	La función debe poder hacerse deliberadamente lenta y costosa en memoria.	Evaluar miles de millones de candidatos por segundo en GPU o ASIC: la velocidad, virtud en todo lo demás, aquí es el defecto.	Almacenamiento de contraseñas · cifrado de monederos	–

Tabla 2 · Matriz: elemento del blockchain × propiedad requerida

Cada celda dice cuánto depende ese elemento de esa propiedad. **Haz clic en cualquier celda** para leer la consecuencia concreta de que esa propiedad falle en ese elemento.

● crítica: el elemento cae de inmediato ◐ necesaria: degrada o habilita ataques indirectos ○ no esencial aquí

ELEMENTO DEL BLOCKCHAIN	PREIMAGEN	2ª PREIMAGEN	COLISIÓN	PREFIJO ELEGIDO	SIN EXTENSIÓN DE LONGITUD	SALIDA PSEUDOALEATORIA
Firma digital `ECDSA / EdDSA sobre H(m)`	◐	●	●	●	○	◐
Identificador de transacción `TXID = dSHA(tx)`	○	●	●	◐	○	○
Árbol de Merkle y prueba SPV `raíz del bloque · inclusión`	○	●	●	◐	◐	○
Encadenamiento de bloques `prev_hash`	○	●	◐	◐	○	○
Prueba de trabajo `H(bloque‖nonce) < objetivo`	●	○	○	○	◐	●
Compromiso — ocultamiento `c = H(m‖r)`	●	○	○	○	◐	●
Compromiso — vinculación `apertura única`	○	●	●	●	◐	○
Dirección derivada de la clave `HASH160 · 160 bits`	●	●	◐	◐	○	◐
HMAC y derivación BIP32 `monederos jerárquicos`	◐	◐	◐	○	●	●
Almacenamiento de contraseñas `KDF · sal · factor de trabajo`	●	◐	○	○	○	◐
Aleatoriedad pública `commit-reveal · sorteos`	◐	○	◐	●	◐	●
Raíz de estado y prueba de reservas `trie de Merkle · solvencia`	○	●	●	◐	◐	○

CONSECUENCIA

Selecciona una celda de la matriz

Cada combinación de elemento y propiedad describe un ataque distinto. Empieza por la fila de la firma digital y la columna de colisión: es el ejemplo que abre esta sección.

Tabla 3 · Consecuencias detalladas, elemento por elemento

ELEMENTO DEL BLOCKCHAIN	SE SOSTIENE EN	ATAQUE SI ESA PROPIEDAD FALLA	CONSECUENCIA EN EL SISTEMA	CASO DOCUMENTADO
Firma digital `ECDSA / EdDSA sobre H(m)`	Colisión · 2ª preimagen · prefijo elegido	Dos documentos con el mismo digest; la firma emitida sobre uno verifica sobre el otro.	Falsificación y repudio **sin robar la clave privada**; se derrumba el no repudio, que es la razón de ser de la firma.	Certificado de CA falso con MD5 (Sotirov *et al.*, 2008); *Flame* (2012) forjó un certificado de firma de código de Microsoft con colisión de prefijo elegido; SHAttered (2017) exhibió dos PDF con el mismo SHA-1.
Identificador de transacción `TXID = dSHA(tx)`	2ª preimagen · colisión	Dos transacciones distintas con el mismo identificador, o una transacción alterada que conserva su TXID.	Referencias ambiguas entre entradas y salidas; una transacción puede invalidar o suplantar a otra ya confirmada.	CVE-2012-1909 y BIP 30, sobre transacciones *coinbase* duplicadas. No confundir con la maleabilidad de 2014: aquélla es un problema de ECDSA, no del hash.
Árbol de Merkle y prueba SPV `raíz de bloque · inclusión`	Colisión · 2ª preimagen · separación de dominios	Dos listas de transacciones distintas con la misma raíz, o una prueba de inclusión válida para algo que no está en el árbol.	Un cliente ligero acepta como confirmado un pago inexistente; la raíz deja de resumir un único conjunto.	CVE-2012-2459, por la duplicación de la última hoja; la ambigüedad hoja/nodo interno con transacciones de 64 bytes. RFC 6962 la evita prefijando `0x00` y `0x01`.
Encadenamiento de bloques `prev_hash`	2ª preimagen	Reemplazar el contenido de un bloque enterrado conservando su hash.	La inmutabilidad desaparece **sin necesidad de reminar nada**: reescribir la historia deja de costar trabajo.	Ninguno contra SHA-256: es exactamente lo que la propiedad impide. El costo de reescritura del laboratorio 8.2 supone que esta propiedad se mantiene.
Prueba de trabajo `H(bloque‖nonce) < objetivo`	Preimagen parcial · uniformidad	Un atajo estructural para hallar nonces bajo el objetivo sin recorrer el espacio.	El costo del minado se desploma, el ajuste de dificultad deja de medir trabajo real y el ataque del 51 % se vuelve barato.	Se vigilan las roturas por rondas reducidas de SHA-256 como indicador temprano. El doble SHA-256 neutraliza además la extensión de longitud sobre el encabezado.
Compromiso — ocultamiento `c = H(m‖r)`	Preimagen · entropía de r	Invertir c, o simplemente hashear todas las opciones cuando el espacio de m es pequeño y falta r.	El sobre se lee antes de abrirlo: se pierde la equidad del volado, de la subasta a sobre cerrado y del voto comprometido.	Laboratorio 8.1 con la casilla del valor aleatorio desactivada: Bob descubre la elección de Alice en milisegundos.
Compromiso — vinculación `apertura única`	Colisión · prefijo elegido	Hallar $(m', r') \neq (m, r)$ con el mismo c y abrir el compromiso de dos maneras distintas.	El emisor cambia su decisión *después* de haberse comprometido: el compromiso deja de comprometer a nadie.	Ataque de *herding* o «Nostradamus» (Kelsey–Kohno, 2006): sobre una construcción de Merkle–Damgård se publica el digest de una profecía y luego se ajusta el texto, por debajo de 2^n.
Dirección derivada de la clave `HASH160 · 160 bits`	Preimagen · 2ª preimagen · colisión	Hallar un guion de redención o una clave pública que produzca la misma dirección; en P2SH la colisión cuesta 2^{80}.	Fondos gastables por quien no es el dueño; en un contrato multiparte, un participante prepara dos guiones válidos para la misma dirección.	Ese margen de 2^{80} es una de las razones por las que los compromisos modernos usan 256 bits (direcciones de 32 bytes en Taproot).
HMAC y derivación BIP32 `monederos jerárquicos`	Inmunidad a extensión · comportamiento PRF	Forjar etiquetas de autenticación sin la clave si se usó $H(k\|m)$; predecir claves hijas si la salida está sesgada.	Autenticación de mensajes rota y monedero jerárquico completo predecible a partir de una sola clave hija filtrada.	Falsificación de firmas en la API de Flickr (2009) por extensión de longitud sobre $H(\text{secreto}\|m)$. BIP32 usa HMAC-SHA-512 precisamente para evitarlo.
Almacenamiento de contraseñas `KDF · sal · factor de trabajo`	Preimagen · costo deliberado	Recorrer diccionarios y espacios de claves a velocidad de GPU o ASIC contra el volcado robado.	Filtrar la base de datos equivale a filtrar las contraseñas en claro; la sal sólo impide amortizar el ataque entre usuarios.	LinkedIn 2012: 6.5 millones de hashes SHA-1 sin sal. De ahí bcrypt, scrypt y Argon2, y el laboratorio 6.2.
Aleatoriedad pública `commit-reveal · sorteos`	Preimagen · vinculación · PRF	Anticipar el valor comprometido, o *moler* entradas hasta que el resultado derivado favorezca al atacante.	El sorteo, la asignación de turnos o la elección de comité dejan de ser imparciales.	El sesgo del último en revelar es un problema de protocolo, no de la función; se mitiga con funciones de retardo verificable o firmas de umbral.
Raíz de estado y prueba de reservas `trie de Merkle · solvencia`	Colisión · 2ª preimagen · completitud	Dos estados globales distintos con la misma raíz, o una prueba de pasivos que omite hojas.	Un cliente ligero valida un saldo falso; una casa de cambio demuestra reservas que no respaldan sus pasivos reales.	Las pruebas de reservas tipo Merkle exigen además demostrar que ningún saldo es negativo: sin esa condición, la omisión de hojas basta para inflar la solvencia aparente.

Laboratorio 9.1 — De una colisión a una firma falsificada

COLISIÓN ⇒ FALSIFICACIÓN · SIN TOCAR LA CLAVE

Dos pagarés idénticos salvo el monto. El atacante busca un folio para cada uno de modo que ambos tengan el mismo hash truncado a n bits. Después Alice revisa y firma *únicamente* el pagaré honesto — y esa misma firma resulta válida sobre el fraudulento.

Documento honesto: `PAGARÉ. Alice pagará a Bob la cantidad de $1,000.00 MXN. Folio:`

Documento fraudulento: `PAGARÉ. Alice pagará a Bob la cantidad de $1,000,000.00 MXN. Folio:`

Hash truncado a 32 bits · Clave privada de Alice `clave-privada-de-al:` · **Buscar la colisión y falsificar** · Detener

FOLIOS PROBADOS	FOLIOS ESPERADOS = 2^(N/2)	TIEMPO
0	≈ 65,536	—

Cada folio probado evalúa los dos documentos, de modo que la colisión entre ambos conjuntos aparece alrededor de $2^{n/2}$ folios —la cota del cumpleaños de la semana 4, aquí en su versión de dos conjuntos—. La firma es HMAC-SHA-256 con la clave de Alice sobre el digest: un sustituto didáctico de ECDSA. Lo único que importa del esquema real es que la firma *sólo depende del hash*, y eso es cierto en ECDSA, EdDSA y RSA-PSS por igual. Con SHA-256 completo esta búsqueda costaría 2^{128} operaciones; con SHA-1 costó $2^{63.1}$ y se ejecutó de verdad en 2017.

■ Una nota sobre el adversario cuántico

El algoritmo de Grover reduce la búsqueda de preimagen de 2^n a $\approx 2^{n/2}$, y el de Brassard–Høyer–Tapp lleva las colisiones a $2^{n/3}$ pero exigiendo memoria cuántica de acceso aleatorio en una escala que hoy nadie considera realista; bajo modelos con memoria acotada, la ventaja cuántica sobre colisiones casi desaparece. La consecuencia práctica para esta tabla es sencilla: **SHA-256 conserva del orden de 128 bits frente a preimagen cuántica y no es la pieza urgente**. La presión post-cuántica está en la firma —ECDSA cae ante Shor—, no en la función hash; de hecho, los esquemas de firma basados exclusivamente en hash (FIPS 205, SLH-DSA) son la respuesta, no el problema.

EVALUACIÓN

Ocho retos verificables

Cada reto se comprueba solo en tu navegador. Al terminar, exporta tu evidencia en JSON y entrégala; el archivo incluye qué resolviste, con qué valores y a qué hora.

Demuestra que el 50 % de avalancha no basta — SEMANA 1

Mide SHA-256 y luego CRC32 en el laboratorio 1.1. Ambas rondan el 50 %: explica en tu reporte qué indicador las separa y por qué.

Encuentra una colisión con tus propias manos — ✓ RESUELTO

En el laboratorio 2.1, halla una colisión del toy-hash. Sube n hasta que tarde más de 5 segundos y anota los intentos.

Registrado: n=16, 163 intentos

Falsifica un MAC sin la clave — ✓ RESUELTO

Ejecuta la extensión de longitud del laboratorio 3.2 y consigue que la etiqueta forjada coincida con la real.

Registrado: clave de 13 B, relleno de 31 B, sufijo "&admin=si"

Verifica 1.1774·2^{n/2} experimentalmente — SEMANA 4

En el laboratorio 4.2, obtén un error relativo menor al 15 % entre media empírica y predicción teórica.

Exhibe una colisión real de SHA-1 — SEMANA 5

Descarga los dos PDF de shattered.io y cárgalos en el laboratorio 5.2 hasta que aparezca la detección de colisión.

Cuantifica lo que aporta la sal — SEMANA 6

Corre el ataque por diccionario con y sin sal, y compara el número de hashes calculados en cada caso.

Calibra un factor de trabajo — SEMANA 6

Encuentra el número de iteraciones de PBKDF2 que hace que una verificación tarde entre 50 y 250 ms en tu equipo.

Genera y verifica una prueba de Merkle — SEMANA 7

Construye un árbol con al menos 5 transacciones y verifica correctamente una prueba de inclusión.

Rompe un compromiso mal construido — SEMANA 8

Ejecuta el volado sin valor aleatorio y consigue que Bob descubra la elección de Alice.

Mina una cadena completa — SEMANA 8

Mina los cuatro bloques con dificultad 4 o más, luego altera un bloque intermedio y explica la cascada.

Verifica una transacción a ciegas — SEMANA 8

Consigue que el compromiso de Pedersen valide una transacción balanceada y rechace una que no lo esté.

Falsifica una firma con una colisión — SÍNTESIS

En el laboratorio 9.1, halla la colisión entre los dos pagarés con al menos 32 bits y consigue que la firma de Alice verifique sobre el documento fraudulento. Explica en tu reporte qué propiedad falló y por qué la clave privada nunca se comprometió.

Exportar evidencia (JSON) · Reiniciar progreso · Tu nombre `Apellido, Nombre`

ANEXO · ESTADO DEL ARTE, AGOSTO DE 2026

Seguridad efectiva de los algoritmos en uso

Objetivo: situar las funciones hash del curso dentro del conjunto completo de *primitivas desplegadas hoy, y leer para cada una tres cifras que no coinciden: el tamaño de la clave, el costo del mejor ataque clásico conocido y el costo estimado del mismo ataque para un adversario cuántico.*

La columna que importa no es la longitud de la clave sino la **longitud de seguridad**: el logaritmo base dos del costo del mejor ataque publicado. Las dos cifras coinciden en criptografía simétrica y divergen radicalmente en la asimétrica —RSA de 2048 bits ofrece 112 bits de seguridad, no 2048— porque la factorización es subexponencial y la búsqueda de clave no lo es. La última columna del cuadro es la que decide la agenda de migración: donde Shor aplica, el margen desaparece por completo; donde sólo aplica Grover, se pierde la mitad del exponente en el peor caso y bastante menos en la práctica.

■ Tabla 4 · Longitud de seguridad, mejor ataque y costo cuántico

VIGENTE recomendado sin reservas   TRANSICIÓN seguro hoy, con fecha de retiro   ROTO ataque práctico publicado

ALGORITMO	LONGITUD DE SEGURIDAD	ATAQUE MÁS EFICIENTE CONOCIDO	COMPLEJIDAD CLÁSICA	COMPLEJIDAD ESTIMADA CON COMPUTADORA CUÁNTICA	REFERENCIA
CRIPTOGRAFÍA SIMÉTRICA					
AES-128 FIPS 197 VIGENTE	**128** bits	Búsqueda exhaustiva de clave. El único ataque publicado contra la versión completa —*biclique*— ahorra menos de dos bits.	$2^{126.1}$	Grover: 2^{64} iteraciones *en serie*; bajo un límite realista de profundidad el costo sube a $\approx 2^{83}$ puertas.	FIPS 197 · Bogdanov–Khovratovich–Rechberger (2011) · Jaques *et al.*, Eurocrypt 2020
AES-256 FIPS 197 VIGENTE	**256** bits	Búsqueda exhaustiva; *biclique* sobre la versión completa.	$2^{254.4}$	Grover: 2^{128} ideal; $\approx 2^{151}$ con profundidad acotada. Es la opción conservadora para el largo plazo.	FIPS 197 · NIST SP 800-57 Pt. 1 Rev. 5 · Jaques *et al.* (2020)
ChaCha20 RFC 8439 VIGENTE	**256** bits	Criptoanálisis diferencial-lineal sobre versión reducida: no pasa de 7 de las 20 rondas.	2^{256} sobre la versión completa	Grover: $\approx 2^{128}$.	RFC 8439 · Coutinho–Neto, Eurocrypt 2021
3DES / TDEA clave de 168 bits ROTO	**112** y ≈32 por el bloque	Encuentro en el medio para la clave; en la práctica manda *Sweet32*: el bloque de 64 bits produce colisiones dentro de la misma sesión.	2^{112} con 2^{56} de memoria · Sweet32 recupera texto claro con $\approx 2^{36}$ bloques	Irrelevante: ya cae con recursos clásicos.	NIST SP 800-131A Rev. 2 (prohibido desde 2024) · Bhargavan–Leurent, CCS 2016
CLAVE PÚBLICA CLÁSICA — FACTORIZACIÓN Y LOGARITMO DISCRETO					
RSA-2048 PKCS #1 · RSA-PSS / OAEP TRANSICIÓN	**112** bits	Criba general del cuerpo de números (GNFS).	$L_N[1/3,\ 1.923] \approx 2^{112}$ Récord público: RSA-250 (829 bits, 2020, ≈2 700 años-núcleo).	Shor: tiempo polinómico. Estimación de recursos de 2025: menos de un millón de *qubits* ruidosos y menos de una semana.	SP 800-57 Pt. 1 Rev. 5 · Boudot *et al.* (2020) · Gidney, arXiv:2505.15917 (2025)
RSA-3072 equivalente a AES-128 TRANSICIÓN	**128** bits	GNFS.	$\approx 2^{128}$	Shor: polinómico. Alargar el módulo sólo encarece el ataque cuántico de forma cúbica, no lo impide.	SP 800-57 Pt. 1 Rev. 5 · Gidney–Ekerå (2021)
DSA y DH sobre cuerpo finito 3072 bits · FFDHE TRANSICIÓN	**128** bits	NFS para logaritmo discreto, con precómputo amortizable por primo (*Logjam*).	$\approx 2^{128}$ Récord público: 795 bits (2019).	Shor: polinómico, mismo circuito que la factorización.	FIPS 186-5 (retira DSA para generar firmas) · Adrian *et al.* (2015) · Boudot *et al.* (2020)
ECDSA y EdDSA P-256 · Ed25519 TRANSICIÓN	**128** bits	Rho de Pollard paralelizado con puntos distinguidos (van Oorschot–Wiener). No hay nada subexponencial sobre curvas genéricas.	$\sqrt{\pi n}/2 \approx 2^{128}$ Récord público: 114 bits (2017).	Shor sobre grupos: $\approx 2\,330$ *qubits* lógicos y $\approx 1.3 \times 10^{11}$ puertas Toffoli. Más barato que romper RSA-2048.	FIPS 186-5 · RFC 8032 · Roetteler–Naehrig–Svore–Lauter, Asiacrypt 2017
FUNCIONES HASH					
SHA-256 SHA-2 · FIPS 180-4 VIGENTE	**128** colisión **256** preimagen	Cumpleaños genérico. El mejor criptoanálisis dedicado no rebasa 31 de las 64 rondas.	2^{128} colisión · 2^{256} preimagen	Grover: 2^{128} en preimagen. Brassard–Høyer–Tapp daría 2^{85} en colisión, pero exige memoria cuántica de acceso aleatorio inviable; con memoria acotada la ventaja se desvanece.	FIPS 180-4 · SP 800-107 Rev. 1 · Brassard–Høyer–Tapp (1998)
SHA3-256 y SHAKE128 Keccak · FIPS 202 VIGENTE	**128** colisión **256** preimagen	Cumpleaños genérico. El criptoanálisis público no supera 6 de las 24 rondas de Keccak-*f*; la construcción de esponja es inmune a la extensión de longitud.	2^{128} colisión · 2^{256} preimagen	Igual que SHA-2: Grover en preimagen, sin ventaja realista en colisión.	FIPS 202 · SP 800-185
SHA-1 FIPS 180-4, retirada ROTO	**≈63** efectivos	Colisión de prefijo elegido por rutas diferenciales: el atacante fija dos prefijos arbitrarios y construye el sufijo.	$2^{63.4}$, ejecutada en 2020 por ≈45 000 USD de GPU rentada ($2^{63.1}$ en SHAttered, 2017)	No aplica: el ataque clásico ya es asequible.	Stevens *et al.*, Crypto 2017 · Leurent–Peyrin, Usenix 2020 · SP 800-131A Rev. 2
MD5 RFC 1321 ROTO	**<32** efectivos	Colisión diferencial de Wang; con prefijo elegido, falsificación de certificados X.509 (Flame, 2012).	Segundos en una laptop · $\approx 2^{39}$ con prefijo elegido	No aplica.	Wang–Yu, Eurocrypt 2005 · Stevens *et al.* (2009) · RFC 6151
POST-CUÁNTICOS — ESTÁNDARES NIST					
ML-KEM-768 Kyber · FIPS 203 VIGENTE	**cat. 3** ≈192 bits	Ataque primal sobre MLWE: reducción de retícula BKZ con tamizado en el bloque.	core-SVP 2^{183}; estimación refinada $\approx 2^{215}$ puertas	core-SVP cuántico 2^{166}. Shor no aplica: no hay estructura de grupo abeliano oculto que explotar.	FIPS 203 (2024) · especificación de Kyber, ronda 3 (2021)
ML-DSA-65 Dilithium · FIPS 204 VIGENTE	**cat. 3** ≈192 bits	BKZ sobre MLWE (recuperación de clave) y MSIS (falsificación directa).	Sin ataque práctico; margen amplio sobre 2^{128}	El tamizado cuántico sólo baja el exponente del bloque de 0.292β a 0.265β: ganancia modesta, ya contemplada en los parámetros.	FIPS 204 (2024) · especificación de Dilithium
SLH-DSA-128s SPHINCS⁺ · FIPS 205 VIGENTE	**128** bits	Ninguno estructural: la seguridad se reduce por completo a la preimagen y la colisión de la función hash subyacente.	2^{128}	Grover sobre el hash; con profundidad acotada la pérdida real es de unos pocos bits. Es la firma con la hipótesis más conservadora del catálogo.	FIPS 205 (2024) · Bernstein *et al.*, SPHINCS⁺
FN-DSA Falcon · FIPS 206, borrador EN DESARROLLO	**cat. 1 y 5** ≈128 / 256 bits	BKZ sobre retículas NTRU. El riesgo práctico está en el muestreo gaussiano de punto flotante, no en el problema duro.	$\approx 2^{128}$ en el nivel 1	Sin aceleración cuántica conocida más allá de un factor polinómico.	Borrador FIPS 206 · especificación de Falcon
HQC KEM de respaldo, códigos SELECCIONADO	**128 · 192 · 256** bits	Decodificación por conjuntos de información (ISD) sobre códigos cuasi-cíclicos.	$\approx 2^{128}$ en hqc-128	La ISD cuántica reduce el exponente aproximadamente a la mitad en el modelo idealizado; los parámetros ya lo absorben.	NIST IR 8545 (2025); selección del 11 de marzo de 2025, borrador de norma previsto para 2026

Convenciones y advertencias de lectura. (i) La longitud de seguridad es $\log_2$ del costo del mejor ataque publicado, no el tamaño de la clave. (ii) Grover no se paraleliza bien: k máquinas sólo dan una mejora de $\sqrt{k}$, de modo que la cifra $2^{n/2}$ es una cota inferior que ninguna arquitectura razonable alcanza; por eso NIST evalúa con un límite de profundidad de circuito. (iii) Las cifras *core-SVP* de los esquemas de retícula son una cota inferior deliberadamente pesimista —ignoran el número de llamadas al oráculo SVP y el costo de la memoria— y siguen siendo objeto de discusión pública en ambas direcciones. (iv) «No aplica» en la columna cuántica significa que el algoritmo ya cede ante un adversario clásico. (v) El borrador NIST IR 8547 propone declarar obsoletos los 112 bits clásicos después de 2030 y prohibirlos después de 2035, lo que fija la fecha real de migración de RSA-2048 y de las curvas de 256 bits.

La lectura conjunta de las cuatro secciones de la tabla explica la asimetría de la transición post-cuántica y confirma lo dicho al cerrar la sección anterior: **la presión está en la clave pública, no en el hash**. AES-256 y SHA-256 sobreviven al adversario cuántico con la mitad de su margen; RSA, DSA y las curvas elípticas no sobreviven en absoluto. Y la respuesta más conservadora que ofrece el catálogo de NIST —SLH-DSA— no es otra cosa que una firma construida enteramente con funciones hash, es decir, con la única primitiva de este curso que el algoritmo de Shor no toca.

Funciones Hash — laboratorio del curso BlockChain. Material de José de Jesús Ángel Ángel · MathCon / Lab-IA, Facultad de Ingeniería, Universidad Anáhuac México. Basado en los apuntes del curso; las implementaciones de SHA-256, SHA-1, MD5, Keccak, CRC32 y HMAC son didácticas, escritas en JavaScript y verificadas contra los vectores de prueba de NIST y RFC. No usar en producción.

Todo el cómputo ocurre en el navegador: ningún mensaje, archivo ni contraseña sale de tu equipo. Referencias: Menezes–van Oorschot–Vanstone, *Handbook of Applied Cryptography*; Katz–Lindell, *Introduction to Modern Cryptography*; NIST FIPS 180-4 y FIPS 202; Nakamoto (2008); Antonopoulos, *Mastering Bitcoin*.